CS 486/686
From Prompting to Fine-Tuning and LoRA

Yuntian Deng

Lecture 22

How to adapt and augment a language model

Learning goals

- Choose the right adaptation strategy.
- Explain **RAG** and **tool use**.
- Explain **SFT**, preference tuning, and **LoRA**.
- Compile and test a reusable **fuzzy function**.

From L21 to L22

A fixed model plus a prompt can do a lot.

```
CS486 helper: Is Assignment 2 due before or after
Chat 9?
```

What if the answer needs fresh course facts,
a calculation, or a durable behavior change?

Running task: CS486 course helper

You have used this helper all term. It is **not** one prompt or one monolithic model.

fresh evidence

course facts + Piazza

fuzzy decisions

route, select, validate

exact control

links, branches, caching

At the end, we will open the real system.

The adaptation ladder

1. better prompt
2. retrieved context (RAG)
3. tools / actions
4. supervised fine-tuning
5. LoRA / adapters

Use the cheapest lever that solves the problem.

Diagnose before choosing the method

Need style?
prompt or SFT

Need fresh/private facts?
RAG

Need calculation/action?
tools

Need durable behavior?
SFT or LoRA

Prompting changes context, not weights

general

Explain gradient descent.

targeted

Explain gradient
descent to a 10-year-old.

A prompt can steer behavior, but cannot add missing knowledge reliably.

Prompt templates package behavior

```
You are a concise CS486 TA. Answer using only
course material when possible. Cite the source for
deadlines.
```

role CS486 TA **task** answer **constraint** cite or decline **format** short + evidence

The template steers behavior, but does not change weights.

When prompting is the wrong fix

missing facts

not in context

tool needs

must calculate or act

brittleness

phrasing changes behavior

RAG motivation

Question: **When is Assignment 2 due?**

A base model does not know this course webpage.

Retrieve evidence first, then answer.

RAG in one sentence

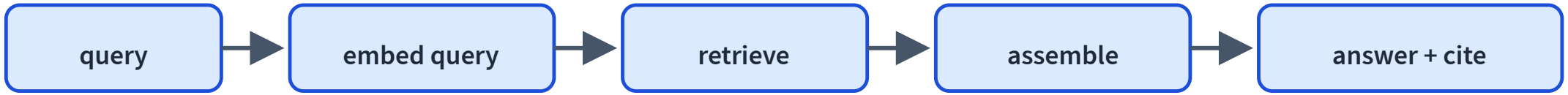
Retrieve relevant documents at inference time, insert them into the prompt, then generate an answer grounded in those documents.

RAG changes the context, not the model weights.

RAG: offline indexing

RAG quality starts before the user asks a question.

RAG: online query path



RAG on real course facts LIVE

Embed the question, retrieve the nearest course chunks, then ground the answer.

"When is Assignment 2 due?" "When and where is the final exam?"

"How much is each chat assignment worth?" "What is the primary textbook for the course?"

When is Assignment 2 due?

retrieve

announcement	Assignment 2 is due Thursday, July 16, 2026 at 11:59 PM, a one-week extension from July 9.	0.81
schedule	Assignment 1 is due Thursday, June 25, 2026 at 11:59 PM (extended from June 18).	0.75
assignments	Assignment 3 is released July 9 and is due Tuesday, August 4, 2026 at 11:59 PM.	0.74

```
System: answer using only the context; cite the source.
Context [announcement]: Assignment 2 is due Thursday, July 16, 2026 at 11:59 PM, a one-week extension from July 9
```

Embeddings reappear

Documents and queries become vectors.

Nearest vectors are retrieved.

This is L18's embedding geometry doing useful work.

Production RAG is usually hybrid

vector

semantic matches

keyword

exact terms

reranker

precision boost

RAG can still fail

wrong chunk

bad retrieval

missing chunk

low recall

ignored evidence

bad generation

RAG reduces hallucination; it does not eliminate it.

Evaluate retrieval and generation separately

context precision

were retrieved chunks relevant?

context recall

did we retrieve enough?

faithfulness

is answer supported?

answer relevance

did it answer?

Some tasks need actions

calculate

weighted grade

execute

run code

lookup

calendar/API

If the model needs to act, give it a tool.

Tool use loop

The model proposes an action; the environment returns evidence.

Tool use, step by step LIVE

The model calls a calculator instead of guessing the arithmetic.

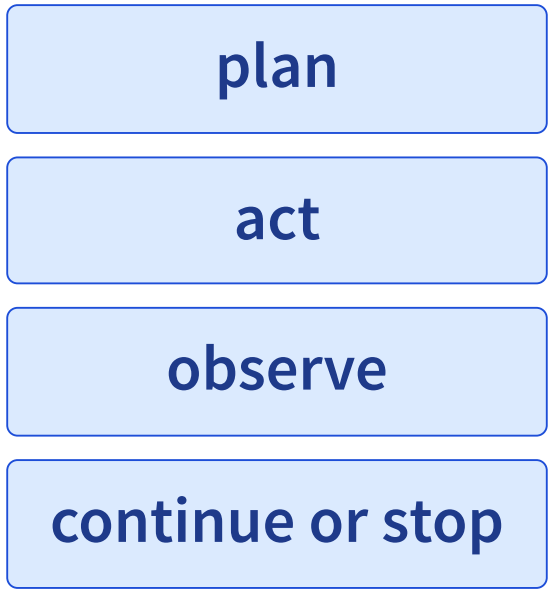
The model proposes an action; the environment (a calculator) returns real evidence; the model uses it.

step reset

user

What is my weighted grade? Assignments 85 (30%), chats 92 (20%), final 78 (50%).

Agents are loops, not magic



Tool use needs guardrails

wrong call

bad action

prompt injection

tool output

attacks prompt

destructive action

needs approval

When should we fine-tune?

Prompting, RAG, and tools change inputs.

Fine-tuning changes weights.

Use it when behavior must change consistently across many examples.

Supervised fine-tuning

Train on curated instruction-response examples.

instruction	ideal response
Answer with citation.	Assignment 2 is due Thu Jul 9 [schedule].
Be concise.	Chat 9 is released Thu Jul 9.

Same next-token loss, targeted dataset.

SFT code sketch

```
for prompt, response in dataset:
    text = chat_template(prompt, response)
    loss = next_token_loss(model, text)
    loss.backward()
    optimizer.step()
```

Pretraining's loss, but on curated behavior examples.

Fine-tuning risks

overfit

small dataset

forget

lose general behavior

artifacts

learn quirks

Use held-out eval and high-quality data.

Preferences tell us what “better” means

Sometimes two answers are plausible, but one is better.

```
(prompt, preferred answer, rejected answer)
```

Demonstrations teach what to say; preferences teach which answer to favor.

Two ways to learn from preferences

RLHF

preferences → reward

model → policy update

Flexible, engineering-heavy

DPO

increase preferred answer

relative to rejected

Simpler, no separate reward-model loop

Full fine-tuning can be expensive

A large model contains huge weight matrices.

Updating all of them is costly.

Can we learn a small update instead?

LoRA learns a low-rank update

$$W' = W_0 + \Delta W$$

$$\Delta W = BA, \quad r \ll d$$

Freeze W_0; train the small matrices A and B.

LoRA forward pass

$$h = W_0 x + \frac{\alpha}{r} B A x$$

W_0: frozen base weight

A, B: trainable adapter matrices

r: adapter rank

α: scaling factor

LoRA: a low-rank update LIVE

Drag the rank: a small BA update, a fraction of the parameters.

Freeze the big weight W_0; train only a low-rank update $\Delta W = BA$. Higher rank = more expressive but more parameters.

rank r **4**

trainable: **192** vs full **576** (rank r=4, d=24)

only **33%** of the weights are trained

Why LoRA is practical

small

few trainable parameters

cheap

less memory/storage

modular

swap or merge adapters

LoRA code sketch

```
config = LoraConfig(
    r=8,
    target_modules=["q_proj", "v_proj"],
)
model = get_peft_model(base_model, config)
train(model, dataset)
```

People normally collect the dataset and train each adapter.

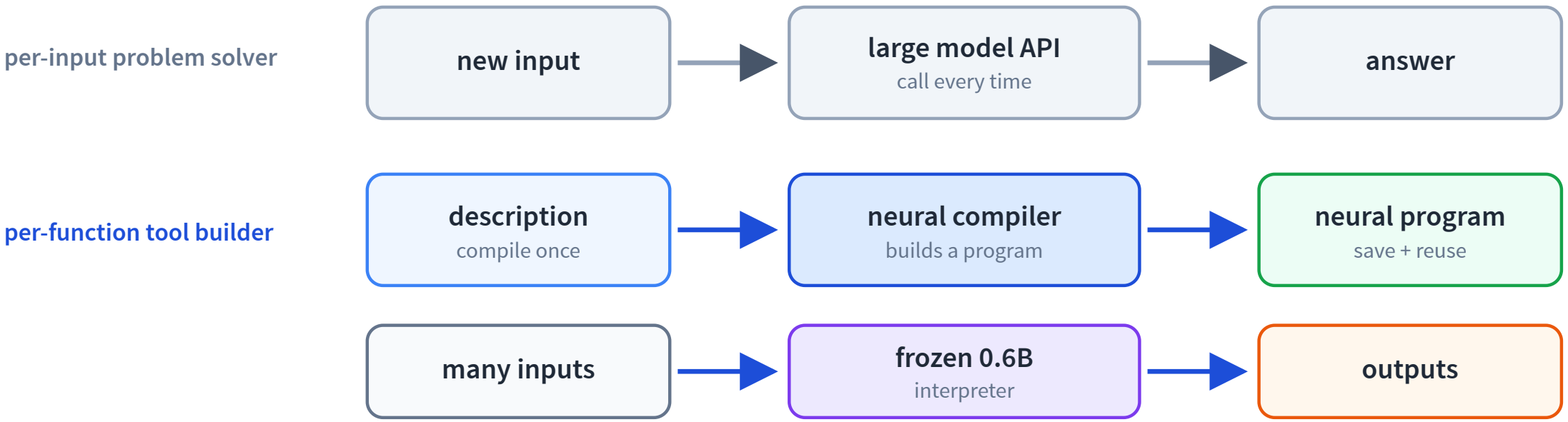
Could another model build the adapter from a description?

Some functions are fuzzy

"Does this message require immediate attention?"

Need your signature by EOD. **urgent**	Newsletter: spring picnic details. **wait**	Whenever you have time—the server is acting odd. **?**
rules become brittle	**large LLM** solves every call again	**fuzzy function** is narrow + reusable

From problem solver to tool builder



When a task repeats, build a reusable tool instead of solving it again.

A PAW program has two parts

$$p = (p_{\text{discrete}},\ p_{\text{continuous}})$$

discrete text

pseudo-program

clean description + examples

normalizes a noisy spec

+

continuous weights

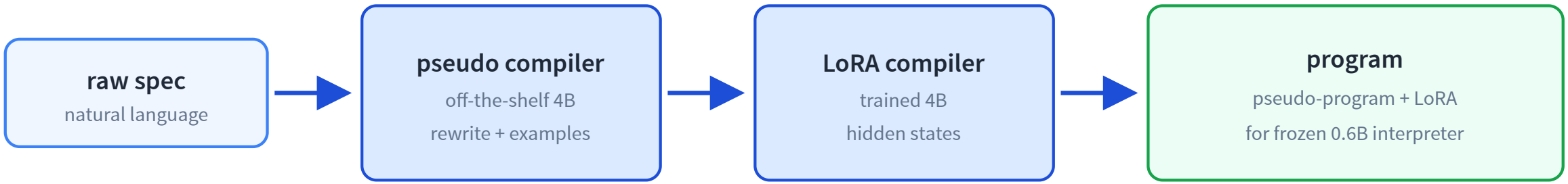
task-specific LoRA

small update to
the frozen LM

fine-grained behavioral control

Qwen3-0.6B from L21 becomes the shared interpreter; programs are hot-swappable.

How can a model compile weights?



The compiler generates A and B; the interpreter never trains.

Train the compiler, freeze the interpreter

FuzzyBench
10M triples (s, x, y)

→

compiler
$s \mapsto$ LoRA

→

frozen 0.6B
$(\mathrm{LoRA}, x) \mapsto \hat{y}$

→

next-token loss
compare $\hat{y}$ with y

Gradient flows back through the frozen interpreter into the compiler.

Standard (default) predicts a LoRA in ~5–10 s

Finetuned Standard synthesizes examples + trains the LoRA

Compile your own fuzzy function LIVE API

Describe once, compile once, then run several inputs with your own program ID.

1 describe → 2 compile → 3 run

Describe the function

```
Classify support ticket urgency. Return ONLY one of: low,
medium, high.

Input: Can you add dark mode to the settings page?
Output: low

Input: The app crashes every time I upload a photo.
Output: high

Input: I was charged twice on my last invoice.
Output: high
```

Compile my function | New function

program ID `not compiled yet`

Run your compiled function

Example 1 | Example 2 | Example 3

```
The checkout page is down for every customer.
```

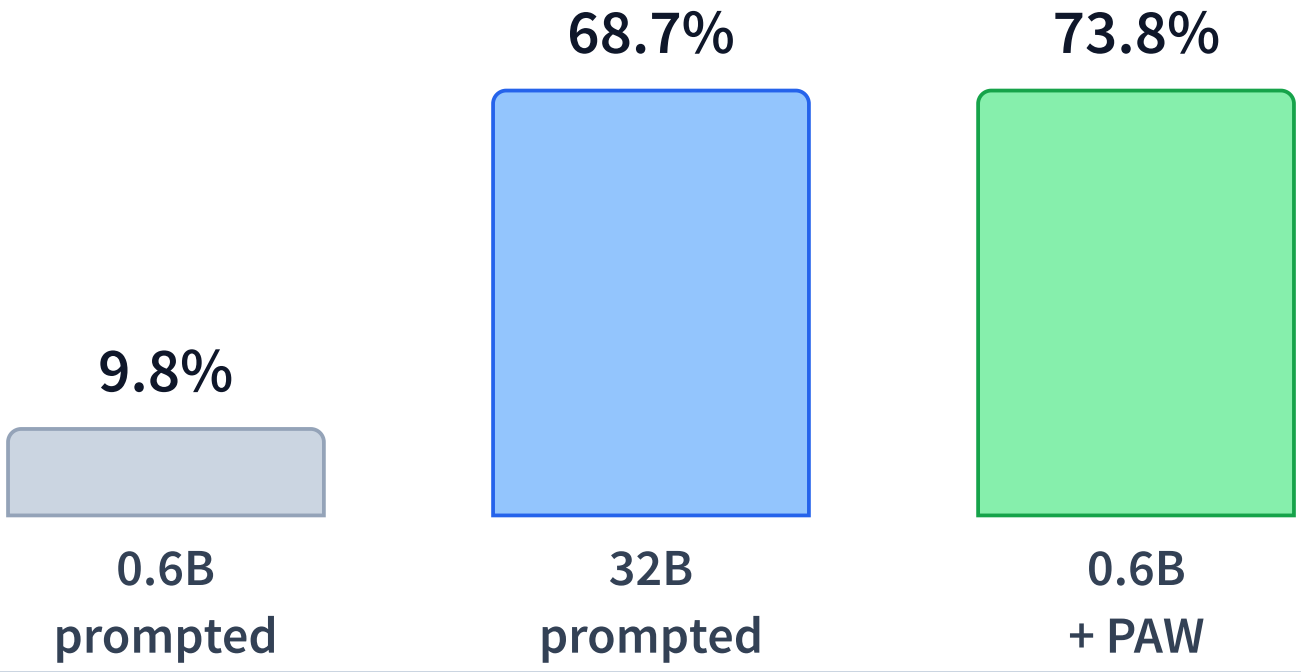
Run this input

Compile the spec, then run more than one input.

Class demo: hosted compile + hosted inference for zero setup. Do not submit private or sensitive text. The resulting program can also be downloaded and run locally.

Edit the examples or labels, then compile your own function.

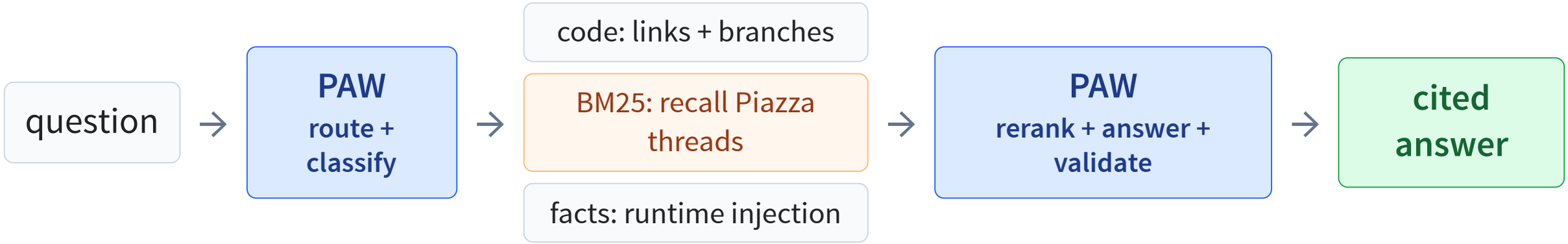
Can the tiny interpreter actually work?



~50× less bf16 inference memory than the prompted 32B model.

Verified FuzzyBench exact match; one shared 0.6B base + about 23 MB per program; ~30 tok/s on an M3 in the reported quantized setup.

The real course helper is hybrid software



30 compiled | 28 serve live | one frozen 0.6B | Finetuned Standard

log monitoring | Avatar Director | Alien Taboo | paper

Use PAW at fuzzy seams

exact rule
ordinary code

fresh knowledge
retrieval / RAG

external action
tool + guardrails

repeated fuzzy text function
PAW

one-off broad reasoning
general LLM

PAW functions are stateless text-in/text-out programs with a shared context budget. Neural outputs remain probabilistic: validate high-stakes results.

System safety checklist

verify context

do not trust
retrieved text blindly

cite sources

show uncertainty

approve actions

human for
irreversible steps

What comes next?

Text-only LMs are one kind of model.

How do models answer questions about images?

L23: Dissecting a Vision-Language Model.

Recap

- ✓ Prompting changes input, not weights.
- ✓ RAG adds external knowledge at inference time.
- ✓ Tools let the model act or compute.
- ✓ SFT changes weights with demonstrations.
- ✓ LoRA makes weight changes small and modular.
- ✓ PAW compiles fuzzy functions into reusable neural programs.

Foundation model: per-input problem solver → per-function tool builder.